Ner 命名实体识别任务

摘 要

本文基于 Qwen2.5-0.5B-Instruct 预训练模型，研究中文命名实体识别（NER）任务中通过参数高效微调降低训练时间的可行性。以 CLUENER 数据集为实验对象，将传统序列标注形式的 NER 转化为生成式 JSON 结构化输出，并在该框架下对不同训练与推理策略进行对比。

在基线模型基础上，引入 LoRA 微调方法，并结合提示词约束与训练流程优化，构建了 v2、v4、v5、v6 等多个模型版本。实验结果表明，v4 模型在保持较高 Precision 的同时显著降低了训练计算开销，体现了训练效率与性能之间的权衡关系。针对生成式方法在严格实体匹配评估下召回率下降的问题，本文进一步尝试了 BIO 序列标注思路进行对比分析。整体结果验证了参数高效微调在资源受限条件下提升训练效率的有效性。

关键词：命名实体识别；生成式模型；LoRA 微调；训练效率；Qwen2.5

目录

一、 引言 3
二、 模型与方法 3
2.1 Baseline 方法 3
2.2 数据准备 4
2.3 模型训练（Qwen2.5-0.5B-Instruct + LoRA 微调） 4
2.4 模型测试 6
三、 实验与分析 7
3.1 数据统计分析 7
3.2 最优模型 v4 的性能分析 7
3.3 不同模型版本的性能与效率对比 8
3.4 训练过程与收敛情况分析 8
四、 讨论与局限性 9
4.1 误差分析 9
4.2 优缺点总结 10
4.3 局限性 10
五、 结论与展望 11
六、附录 11
6.1 各版本实验参数对照表 11
6.2 测试集预测结果说明 12
6.3 项目代码框架 12

一、引言

命名实体识别（Named Entity Recognition，NER）是自然语言处理领域中的一项基础任务，其目标是从非结构化文本中识别并抽取具有特定语义类别的实体信息，如人名、机构名、地点名等。NER 作为信息抽取、知识图谱构建、问答系统和搜索引擎等应用的重要前置环节，其性能直接影响下游任务的效果。

随着预训练语言模型的发展，基于大规模语料训练的指令微调模型在多种自然语言处理任务中表现出较强的泛化能力。然而，在具体下游任务中对大模型进行微调时，往往面临训练成本高、训练时间长以及计算资源受限等问题，因此，在保证模型性能的前提下，探索更高效的大模型训练与部署方式具有实际意义。

本项目**基于 CLUENER 细粒度中文命名实体识别数据集**开展实验研究。该数据集包含地址（address），书名（book），公司（company），游戏（game），政府（government），电影（movie），姓名（name），组织机构（organization），职位（position）和景点（scene）共 10 类实体，**实体类别细粒度高、分布不均，**对模型的语义理解和实体边界识别能力提出了较高要求。相较于传统基于序列标注的 NER 方法，本文**采用生成式建模方式**，将命名实体识别任务视为条件文本生成问题，引导模型直接生成结构化的实体抽取结果。

本项目选用 Qwen2.5-0.5B-Instruct 作为基础模型，对其进行下游任务微调训练。项目的主要目标是**在保证命名实体识别精确率基本稳定的前提下，通过合理的训练策略和参数高效微调方法，尽可能减少模型训练时间和计算开销，**从而验证大模型在资源受限环境下完成细粒度命名实体识别任务的可行性与实用性。

二、模型与方法

2.1 Baseline 方法

本项目以 Qwen2.5-0.5B-Instruct **的全参数生成式微调方法**作为**基线模型**，将命名实体识别任务建模为指令引导下的结构化 JSON 生成任务，训练过程中对模型全部参数进行更新并采用自回归语言模型损失进行优化，该方法实现简单但训练时间与计算开销较高，作为后续参数高效微调策略在训练效率与性能上的对照基线。

2.2 数据准备

2.2.1 数据来源与任务定义

本项目使用 CLUEBenchmark 的 CLUENER 数据集开展 NER 任务。每条样本由文本(text)与标注(label)构成,目标是抽取 10 类实体:address, book, company, game, government, movie, name, organization, position, scene。

2.2.2 标注格式转换（convert.py）

原始 CLUENER 数据的 label 采用“实体 → 位置 span”形式表达，适合序列标注/抽取式建模；但本项目采用生成式 JSON 输出，因此需要把标注转换为**“实体类型 → 实体字符串列表”**的目标格式，使其可直接作为模型生成目标。

2.2.3 训练集 token 分布统计（show_tokens.py）

生成式训练需要**设置 max_source_length 与 max_target_length**。若设置过小会造成截断丢信息，过大会导致大量样本在末尾被填充无效的 padding token，会引入显著的冗余计算，增加训练时间和显存开销。为确定合适的数值，统计了训练集输入文本和输出标签的 token 数范围与均值。

2.3 模型训练（Qwen2.5-0.5B-Instruct + LoRA 微调）

2.3.1 训练总体思路

本项目采用 Qwen2.5-0.5B-Instruct 作为基座模型，通过 **LoRA（PEFT）进行参数高效微调**:只训练注意力投影层的低秩增量参数,降低显存占用并加快训练。

训练目标：给定输入文本，让模型生成符合约束的**单行 JSON** 实体结果。

2.3.2 训练数据构造（dataset.py）

(1) 系统提示词与标签集合

- LABEL_LIST：定义允许的实体类别集合（10 类），用于约束任务空间与后处理过滤。
- SYSTEM_PROMPT：规定输出必须为单个 JSON（或 {}），实体必须为原文连续子串、按出现顺序去重、不确定不输出等。

该提示词**在训练与推理阶段都保持一致**，使模型学习稳定的输出格式与抽取规则。

(2) NerDatasetV2：样本组织与监督策略优化

在基线实现中，训练样本采用将指令提示与目标输出直接拼接后统一送入模型进行训练的方式。这种实现虽然简单，但会在训练过程中引入额外的无效监督：模型不仅需要学习从文本生成实体标注结果，还会被迫对指令提示本身进行语言建模，从而分散对实体抽取任务的学习重点。

为解决上述问题，本文在优化模型中引入了改进的数据构造与监督策略（NerDatasetV2）。该策略在保持任务输入输出形式不变的前提下，对样本内部结构进行了显式划分，将**指令与输入文本**作为上下文条件，将**实体标注结果**作为唯一需要被监督学习的目标输出。在损失计算阶段，仅**对模型生成实体标注结果对应的 token 计算损失**，而不对指令提示内容施加约束。

通过这种方式，模型在训练过程中不再消耗学习能力去拟合固定的任务指令，而是将参数更新集中于实体抽取这一核心任务，有效提高了监督信号的利用效率，并为后续的参数高效微调提供了更加稳定的训练基础。

同时，使用 **cache_tokenized 进行缓存加速**，为 True 时将 token 化后的样本缓存到内存，减少每轮的重复 tokenization 时间，**提高训练速度**。

(3) DataCollatorForCausalLM：动态 padding 与批处理效率优化

在生成式训练场景下，不同样本之间的输入长度和输出长度差异较大。若采用固定最大长度对所有样本进行填充，会导致大量无效 padding token 的引入，从而增加前向计算与显存占用，降低整体训练效率。

针对上述问题，模型采用了**基于批次的动态填充策略**。在样本级别仅进行必要的截断，将**填充操作**延迟至批处理阶段，由数据整理模块根据当前批次中最长样本动态补齐序列长度。这种方式能够减少无效 padding 的比例，使模型计算集中于真实有效的输入与输出 token。

2.2.3 训练主程序（train_v4.py）

(1) 训练设备选择

根据运行环境自动选择可用设备（如 CUDA / MPS / CPU），保证代码在服务器与本地均可运行，减少环境相关修改成本。

(2) 模型与 tokenizer 加载

使用 HuggingFace AutoTokenizer 与 AutoModelForCausalLM 加载基座模型，并设置 pad/eos，确保 batch padding 与生成终止行为可控。

(3) LoRA：参数高效微调与训练流程优化

针对全参数微调耗时极大的问题，模型采用 LoRA（Low-Rank Adaptation）对基座模型进行参数高效微调，在保持预训练参数冻结的前提下，仅在注意力相

关的投影层中引入**低秩可训练参数**。LoRA 的秩大小及缩放系数等参数通过实验经验进行设定，在**模型表达能力与训练效率之间取得平衡**。

通过这种方式，模型在训练阶段仅需更新极少量新增参数，显著降低了反向传播过程中的计算量和显存占用，**有效缩短训练时间**。同时，避免了对原始预训练权重的大规模扰动，在保证模型表达能力的前提下**提升了训练稳定性**。

在训练流程实现上，围绕 LoRA 微调特性对训练过程进行了相应调整，包括**仅对可训练参数进行优化**、在每个训练周期后**基于验证集损失评估模型性能**，并**保存性能最优的模型权重**。保证了在**训练效率提升**的同时，模型性能能够得到稳定监控与可靠对比。

2.3.4 LoRA 合并（merge_lora_v4.py）

推理部署阶段，为减少加载 base 和 adapter 的复杂度，本项目将 best LoRA adapter 合并回基座模型权重，导出单独的 merged_model，便于后续批量推理与复现实验。

2.4 模型测试

(1) 推理 prompt 构造

使用与训练相同的 system prompt + user text，并添加 generation prompt，保证输入形式一致。

(2) 生成策略控制

设置 max_new_tokens、do_sample=False 等参数，提升生成稳定性，限制输出长度，并减少 JSON 解析失败的概率。

(3) JSON 提取与预测规范化

生成式输出可能带多余字符，因此实现两层处理：

· extract_first_json_obj()：从输出文本中提取第一个完整 JSON 对象，避免尾部噪声影响

· normalize_pred()：过滤非法类型、统一值为 list、去空去重，并对过短、易误报实体进行规则约束，提高评测一致性与鲁棒性

最终每条样本写出 text 与 pred_json（或 {}），形成标准预测文件。

三、实验与分析

3.1 数据统计分析

在模型训练之前，本文对训练集中输入文本及对应实体标注结果的 token 数量分布进行了统计分析，其结果如图 1 所示。

大多数样本的输入与输出长度均远低于模型所能支持的最大序列长度。基于此，实验中为输入文本和输出结果分别设置了合理的最大长度限制，降低冗余计算开销并提升训练效率。

3.2 最优模型 v4 的性能分析

在本实验中，v4 被选为最优模型版本，用于**训练效率优先的设定**下评估模型的命名实体识别性能。图 2 展示了 v4 在验证集上的整体 Micro Precision、Recall 与 F1 指标。

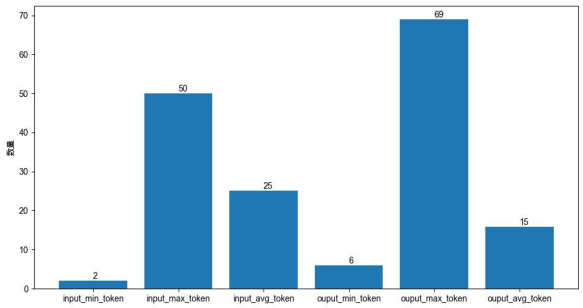


图 1：训练集 token 分布情况

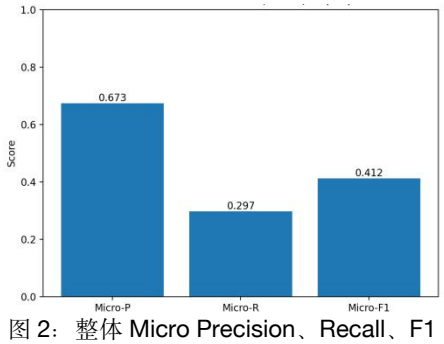


图 2：整体 Micro Precision、Recall、F1

从整体看，v4 在 Precision 指标上表现较为突出，表明模型在实体预测过程中具有**较强的置信度控制能力**；同时，其 Recall 指标相对较低，说明模型在当前设定下对实体抽取范围采取了**较为保守的策略**，使得模型在整体 F1 指标上较为稳定。

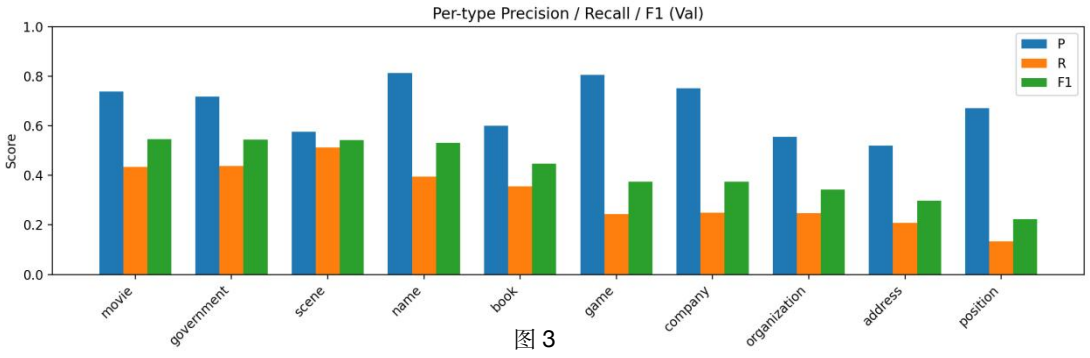


图 3

进一步从不同实体类型的性能分布来看（图 3），v4 在多数类别上呈现出

Precision 高于 Recall 的一致特征，其中 name、movie、government 等实体类型的识别效果相对较好，而 address、position 等类别的召回率偏低。该结果表明，v4 在效率优先的训练目标下，形成了以高置信度预测为主的实体抽取行为模式。

3.3 不同模型版本的性能与效率对比

为评估不同训练与推理策略对模型性能的影响，本文在验证集上对多个模型版本进行了对比实验，相关结果汇总如表 1 所示。

模型版本	Prompt 类型	bad_pred_json	Precision	Recall	F1
v2-weak	弱约束 prompt	0	0.6174	0.6127	0.6150
v2-strong	强约束 prompt	0	0.6268	0.6184	0.6226
v4	强约束 + 训练优化	72	0.6730	0.2968	0.4119

表 1：不同模型版本在验证集上的性能对比

在 v2 系列模型中，通过**引入更强约束的 prompt**，模型在 Precision、Recall 及 F1 等指标上均取得了一定提升，说明**明确且严格的实体抽取规则**有助于引导生成式模型输出更符合任务要求的结构化结果。然而，该类方法仍需要对模型的大量参数进行更新，训练阶段的计算开销较高。

在此基础上，v4 在保持强约束提示词的同时**引入了进一步的训练优化策略**，其 Precision 指标进一步提高，表明模型在实体预测上更加保守；Recall 出现明显下降，导致整体 F1 分数低于 v2 系列模型。说明**训练策略的调整会显著影响模型在精确率与召回率之间的平衡**。

需要注意的是，v4 的性能变化并非单纯追求指标最优，而是以训练效率为主要优化目标。相比 v2 系列，v4 在**减少参与训练参数规模**的同时，显著**降低了训练阶段的计算成本**，在较短的训练时间内完成模型收敛。

3.4 训练过程与收敛情况分析

为进一步分析最优模型 v4 在效率优先设定下的训练稳定性与收敛特性，本文对训练过程中的损失变化情况进行了记录与可视化，如图 4 所示。

在效率优先的训练设定下，v4 的训练过程表现出**良好的稳定性与较快的收敛特性**。**训练集**损失在初期迅速下降，随后整体呈现稳定波动但无明显发散趋势，表明在小 batch 与梯度累积策略下模型仍能够保持有效优化；同时，**验证集**损失在有限的训练步数内持续下降且未出现明显反弹，说明模型在较短训练时间内

已完成有效学习。

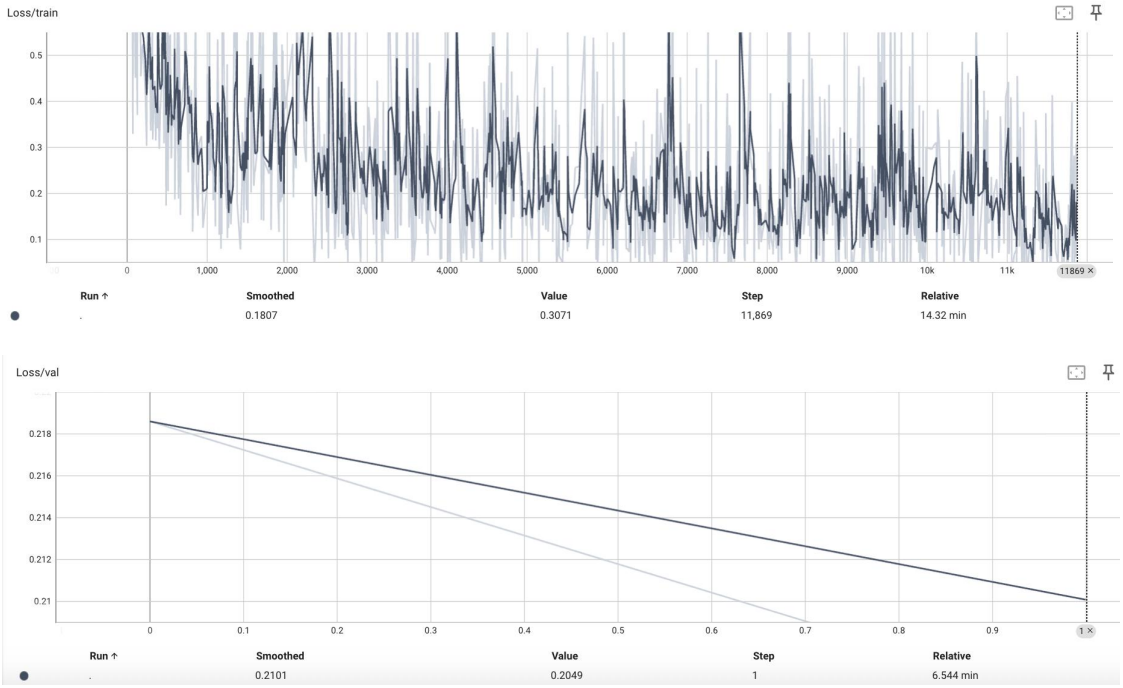


图 4：训练集（上）/验证集（下）损失曲线

此外，结合不同实体类型的 TP、FP 与 FN 分布情况（图 5）可以看出，v4 在多数类别中**对误报进行了有效控制**，而 FN 相对较多，这与其 Precision 较高、Recall 偏低的整体性能表现一致。

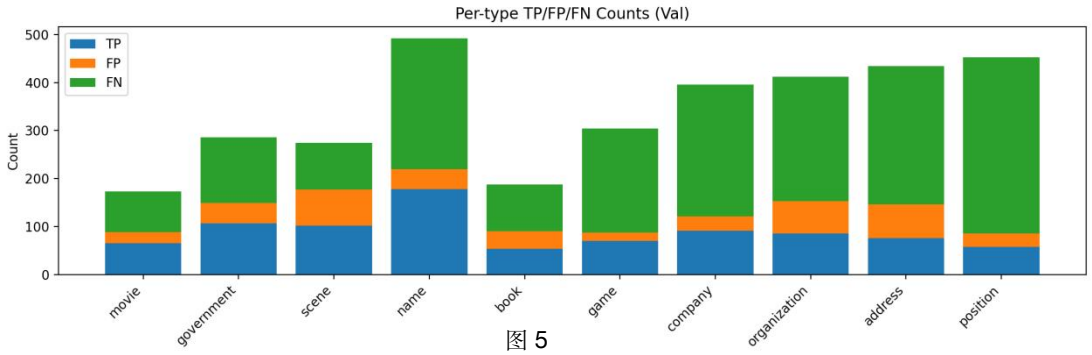


图 5

上述结果表明，v4 能够**在显著降低训练计算开销的前提下实现稳定收敛**，验证了参数高效微调策略在减少训练时间方面的有效性。

四、讨论与局限性

4.1 误差分析

本项目将命名实体识别建模为生成式任务，模型在指令提示下直接生成结构

化 JSON 实体结果。实验迭代表明，模型在**效率优先**的目标下具备较高工程可行性，但误差形态也呈现出生成式方法的典型特征：

一方面，模型输出对**格式约束**和**抽取规则**高度敏感，轻微的**训练侧变化**，例如**样本拼接方式、loss mask 范围、最大长度截断策略**等，都会改变模型学习到的输出行为，导致性能波动；这在 v5、v6 中体现为训练监督信号不稳定后出现明显漏检（FN 增多、Recall 下滑），同时部分样本产生不合规 JSON 输出（bad_pred_json 增加），使得评估阶段无法正确解析或严格对齐，从而进一步拉低整体指标。

另一方面，在严格匹配评估（即实体类型与原文字符跨度需完全一致）的设定下，生成式模型容易出现**边界不精确**与**抽取策略过于保守**两类典型错误。

其中，边界不精确会**直接带来 FN 或 FP**：例如模型能够识别实体的语义范围，但在生成结果中遗漏后缀、截断实体文本，或将泛称误当作完整实体，导致预测结果无法与标注 span 完全对齐，从而在评测中被判定为错误；而抽取策略过于保守则表现为**模型仅输出高置信度**。以 v4 为例，其 Precision 上升说明误报被有效抑制、模型更倾向于输出高置信度实体，但 Recall 明显降低，反映出在效率优先的训练配置下模型形成了更谨慎的抽取策略，从而牺牲了一部分覆盖率。

4.2 优缺点总结

生成式 NER 的优势在于流程简洁、易与大模型指令微调对齐，并可结合 LoRA 显著降低训练成本；但其不足在于对训练监督和输出规范极其敏感，且在严格 span 匹配下更容易因边界细节与保守倾向导致召回受限。

4.3 局限性

本方法的局限性主要体现在生成式 NER 对**输出规范和训练监督较为敏感**，预测结果容易受到边界偏差和格式约束的影响。在严格 span 匹配评估下，轻微的实体截断或遗漏都会被计为错误，从而拉低 Recall。此外，在效率优先设定下，模型更偏向保守抽取，体现出 Precision 与 Recall 之间的取舍。

基于上述局限，曾尝试 **BIO 序列标注思路**作为优化：将任务转为逐 token 分类，直接输出每个 token 的 BIO 标签，从而避免生成式解码带来的额外时间开销。

实验体验上，BIO 在训练与推理速度上更有优势，同时输出形式更稳定，不容易出现“非 JSON/字段不合规”等格式问题。然而，BIO 路线需要对数据标注形态、模型训练目标与评测流程进行较大改造，与生成式代码框架差异较大，因此本文最终仍以生成式方案作为主线，并将 BIO 作为后续可进一步验证的替代方向。

五、结论与展望

本文基于 Qwen2.5-0.5B-Instruct 模型，将命名实体识别任务建模为生成式抽取问题，探索了**在保证基本性能的前提下降低训练时间**的可行性。实验结果表明，引入参数高效微调策略并配合合理的训练配置，可以在较短训练时间内实现稳定收敛。以 v4 为例，模型在 Precision 上表现较好，体现了效率优先设定下对误报的有效控制，尽管 Recall 有所下降，但整体性能仍保持在可接受水平。

未来工作可进一步围绕生成式 NER 的稳定性与召回率展开优化，例如改进监督方式或输出约束策略；同时，可结合 BIO 序列标注方法在速度和边界稳定性方面的优势，探索不同任务场景下更合适的实现方案。

六、附录

6.1 各版本实验参数对照表

参数项	v2	v4	v5	v6
微调方式	LoRA	LoRA	LoRA	LoRA
max_source_length	256	256	256	256
max_target_length	160	160	160	160
epochs	1	2	3	2
batch_size	2	2	2	2
grad_accum_steps	8	8	8	8
learning_rate	2e-4	1e-4	8e-5	1e-4
LoRA rank (r)	8	8	16	8
LoRA alpha	16	16	32	16
LoRA dropout	0.05	0.05	0.05	0.05
LoRA target_modules	q/k/v/o_proj	q/k/v/o_proj	q/k/v/o_proj	q/k/v/o_proj
warmup / scheduler	无	无	有	无
gradient clipping	无	无	有	无
数据采样策略	原始数据	原始数据	原始数据	过采样
主要实验目的	基础 LoRA 训练	效率优化	稳定性探索	数据分布影响

6.2 测试集预测结果说明

本实验中,模型在测试集 raw_data/test.json 上的预测结果已通过脚本 inference.py 生成，并以 JSONL 格式保存为 pred_test.jsonl。

6.3 项目代码框架

NLP_qwen_ner/
├─ merged_model_v4/ # 最终可推理模型（基座+LoRA 合并后的权重）
├─ data/ # 预处理后的训练/验证数据
├─ raw_data/ # 原始数据
├─ **eval_outputs/** **# 评估输出（图表、统计结果等）**
├─ bio/ # BIO 序列标注探索实验（尝试，不作最终方案）
│
├─ **convert.py** **# 数据格式转换/清洗**
├─ **dataset.py** **# 训练数据构造**
├─ train_v2.py # v2 训练脚本
├─ **train_v4.py** **# v4 训练脚本（最终方案：效率优先的训练优化配置）**
├─ train_v5.py # v5 训练脚本
├─ train_v6.py # v6 训练脚本
├─ merge_lora_v4.py #合并 LoRA 到基座：得到 merged_model_v4
│
├─ **inference.py** **# 测试集推理脚本：输出 pred_test.jsonl**
├─ **inference_val_v4.py** **# 验证集推理脚本：输出 pred_val.jsonl**
├─ eval_val.py # 评估脚本
│
├─ show_tokens.py # Token 长度统计
├─ pred_val.jsonl # 验证集预测结果
└─ **pred_test.jsonl** **# 测试集预测结果**